

CONTRATO DA API E AUTENTIFICAÇÃO

Projeto: Simbionte – Simulação de Ecossistema de Influência Digital

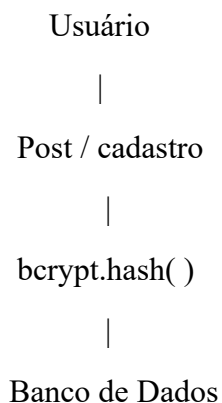
Subgrupo D: Segurança e Regras (Ana Luiza, Vitor e Yasmin)

1. Objetivo

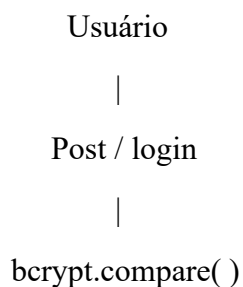
Este documento define o contrato da API responsável pelo processo de autenticação do Projeto Simbionte. O objetivo é padronizar a comunicação entre o front-end (Subgrupo B), o back-end (Subgrupo A) e a equipe de testes de integração (Subgrupo D), estabelecendo o formato exato das requisições, respostas e protocolos de segurança das rotas.

2. Visão Geral: Fluxo de Autenticação

2.1 Cadastro



2.2 Login



Senha correta?	
____ ____	
Sim	Não
JWT	Erro 401

3. Endpoints Públicos (Sem Autenticação)

O processo de autenticação seguirá boas práticas de segurança, garantindo a proteção das credenciais dos usuários e o controle de acesso às rotas do sistema.

3.1. Rota de Cadastro

Responsável pelo registro de novos usuários no ecossistema.

- **Método:** POST
- **Endpoint:** /cadastro

Corpo da Requisição (Request Body):

Campo	Tipo	Obrigatório	Descrição
nome	string	Sim	Nome de exibição do usuário
email	string	Sim	E-mail válido e único no sistema
senha	string	Sim	Senha em texto plano (será processada pelo back-end)

Exemplo de Payload (JSON):

```
{  
  "nome": "João Silva",  
  "email": "joao@email.com",  
  "senha": "MinhaSenha123!"  
}
```

Respostas Esperadas:

Status Code	Cenário	Resposta (JSON)
201 Created	Sucesso	{"mensagem": "Usuário cadastrado com sucesso."}
400 Bad Request	e-mail já existente ou erro de validação	{"erro": "E-mail já cadastrado ou dados inválidos."}

3.2 Rota de Login

Responsável por validar credenciais e iniciar a sessão.

- **Método:** POST
- **Endpoint:** /login

Corpo da Requisição (Request Body):

Campo	Tipo	Obrigatório	Descrição
email	string	Sim	E-mail cadastrado no sistema
senha	string	Sim	Senha em texto plano para validação

Exemplo de Payload (JSON):

```
{
  "email": "joao@email.com",
  "senha": "MinhaSenha123!"
}
```

Respostas Esperadas:

Status Code	Cenário	Resposta (JSON)
200 OK	Sucesso - (Credenciais válidas)	{"token": "eyJhbGciOi...", "tipo": "Bearer"}
401 Unauthorized	Falha - (E-mail ou senha incorretos)	{"erro": "Credenciais inválidas."}

4. Endpoints Privados (Protegidos)

Qualquer rota que interaja com o ecossistema interno (ex: dashboard, criação de agentes, verificação de *TrustScore*) exigirá autenticação ativa.

O algoritmo *bcrypt* foi escolhido por gerar hashes unidirecionais utilizando salt automático, aumentando a resistência contra ataques de força bruta e tabelas rainbow.

4.1. Regra de Acesso

O cliente (front-end) deve, obrigatoriamente, enviar o token JWT recebido no ato do login através do cabeçalho da requisição HTTP.

Formato do Cabeçalho (Header):

- **Key:** Authorization
- **Value:** Bearer <token_jwt>

4.2. Tratamento de Exceções de Sessão

Caso o acesso a uma rota privada seja feito de forma inadequada, o servidor retornará os seguintes erros:

Satus Code	Cenário	Resposta (JSON)
401 Unauthorized	Token ausente ou formato inválido	{"erro": "Acesso não autorizado. Sessão inválida."}
403 Forbidden	Token expirado ou sem permissão	{"erro": "Sessão expirada ou permissão negada."}

5. Diretrizes de Segurança

Para garantir a integridade dos dados, a arquitetura do back-end obedece às seguintes regras irrevogáveis:

- **Hashing:** A senha do usuário **nunca** será armazenada em texto puro no banco de dados.
- **Criptografia:** O módulo *bcrypt* (com custo de processamento/*saltRounds* = 10) será utilizado para gerar o hash unidirecional.

- **Validação:** A validação de login ocorrerá exclusivamente através da função assíncrona *bcrypt.compare()*.
- **Sessão:** A manutenção do estado e identificação do usuário utilizará **JSON Web Tokens (JWT)**.

6. Observações Finais

- Todas as requisições e respostas de corpo utilizarão o formato *application/json*.
- As mensagens de resposta seguirão rigorosamente o padrão de **Status Codes** do protocolo HTTP.